

2026-08-18-3rd-forced-logout

Incident 2026-08-18 — 3rd forced-logout of milosvasic user session

Revision: 1 **Last modified:** 2026-08-18T23:59:00+02:00

Note on tracking ID: This incident is tracked as workable-item **BOB-120** in the SSoT DB. It is the 3rd occurrence of this incident class on this host, following the §12.12-anchor-producing incident of 2026-07-07 and BOB-116 (2026-08-18, ~20:50:59).

Reporter: session-continuation task dispatch (2026-08-18, following the operator’s earlier CRITICAL reports of the same class this same day) **Session-under-investigation:** 4db6eadb-03e7-466b-9cb4-34b2b2bd30f3 (boba SDD orchestration) **Investigator:** boba autonomous loop, systematic-debugging Phase 1 **Class:** perceived host power event (CONST-033 Operational Note class) — 3rd occurrence **Terminal state:** partial root cause with §11.4.6 UNCONFIRMED boundary (same class as BOB-116); **NEW architectural finding:** the preventive resource-pressure monitor landed for BOB-116 (task #77) runs *inside* the very user@1000.service scope it exists to protect, so it dies together with everything else at the moment of the kill and cannot have fired in the minutes immediately preceding it.

Timeline (2026-08-18, Europe/Belgrade, CEST +02:00)

Time	Event	Evidence
11:08	Host boot (host has not rebooted since — uptime shows 12:46h	uptime, who -b

Time	Event	Evidence
	continuous at time of writing)	
20:50:59	1st kill of the day — user@1000.service SIGKILLed (BOB-116, documented separately in docs/incidents/2026-08-18-perceived-forced-logout-2nd.md)	prior incident doc
20:51:16	New session (session 18) begins post-BOB-116	journalctl
22:42:40 → 22:42:42	boba-resource-pressure-check.service fires + completes (PASS, no signature tripped)	journalctl --user -u boba-resource-pressure-check.service
22:57:48 → 22:57:58	boba-resource-pressure-check.service fires 3 more times in rapid succession (22:57:48, 22:57:51, 22:57:55 — all under the same systemd[1665349] user-manager instance, i.e. no session restart occurred between these fires) — all complete	same
23:38- 23:45	qbittorrent-proxy container's health-probe socketserver logs a ConnectionResetError: [Errno 104] Connection reset by peer roughly every 30s, continuously, right up to the kill (last entry 23:45:44.737 , five seconds before the kill)	docs/qa/BOB-120/ qbittorrent_proxy_socketserver_traces.log
23:45:49	2nd kill of the day / 3rd overall on this project — user@1000.service Main	docs/qa/BOB-120/ journalctl_23-42_to_23-46.log

Time	Event	Evidence
	process exited, code=killed, status=9/KILL. Cascade SIGKILL of every user process (gnome-shell, Xwayland, mutter-x11-fram, gvfsd/gvfsd-recent, ssh, dbus-daemon, gnome-keyring-d, ibus-daemon/x11, at-spi-bus-launcher/at-spi2-registryd, gsd-media-keys/datetime/keyboard/ally-settings/sharing/screensaver/printer, chrome_crashpad, Telegram, jockett, qbittorrent-nox, boba-jockett python PID 3111634, common PID 2319406, plus every MCP-server / node / npm / claude process in-session) — 73 Killing process lines total in the captured journal window	
23:45:49	gdm-password session closed for milosvasic; systemd-logind removed session 19	same
23:45:50	GDM greeter session (23/24) spawned	same
23:45:52	All four boba-stack containers (qbittorrent, jockett, boba-jockett, qbittorrent-proxy) show a fresh CreatedAt of 23:45:52 — i.e. they were re-created 3 seconds after the kill, before the operator's session had even returned	podman ps -a --format "{{.Names}}\t{{.CreatedAt}}\t{{.Status}}" (captured live)
23:49:00		same

Time	Event	Evidence
23:49:00	New GDM password session opened for milosvasic (session 25); new systemd --user manager instance (PID 3115149) starts boba-resource-pressure-check.timer starts (re-enabled by the fresh user-manager boot) but shows Trigger: n/a and systemctl --user list-timers reports NEXT: - (no next-fire time computed) at time of writing, ~7 minutes into the new session	docs/qa/BOB-120/timer_fire_history.log
~23:52 onward	Investigation + this incident document authored	this session

3 subagents were mid-flight at kill time: the BOB-117 partial-fix recovery (its ssh push process, PID 1839991, appears in the 73-line kill list above), a BOB-083-related retry stream, and this incident-write-up dispatch. The BOB-117 partial fix was successfully recovered post-restart and committed as 111e584 fix(rutracker,B0B-117): recover partial fix – §11.4.6 upstream_captcha reclassification (from stopped subagent ab2a34107) — this was already the current HEAD commit at the start of this investigation, confirming the recovery held.

CONST-033 triage (mandatory per CLAUDE.md)

Check	Result	Evidence
Uptime discontinuity	No — kernel uptime 12:46h continuous since 11:08 boot	uptime, who -b journalctl -k --since "2026-08-18 00:00:00" \ grep -iE "will suspend\
systemd-suspend markers	None in journal for 2026-08-18	

Check	Result	Evidence
Kernel OOM-killer	None in journal for 2026-08-18	systemd-suspend" — zero matches journalctl -k --since "2026-08-18 00:00:00" \ grep -iE "oom-kill\ killed process" — zero matches
systemd-oomd trigger	No entries for the incident window	journalctl -u systemd-oomd --since "2026-08-18 23:30:00" --until "2026-08-18 23:50:00" — empty
Lid open/close events	None in the incident window (distinguishes this from BOB-116, where a lid re-open at 20:51:06 coincided with the operator's return)	journalctl --since "2026-08-18 23:00:00" --until "2026-08-18 23:50:00" \ grep -iE "lid (open\ clos)" — zero matches
HandleLidSwitch	ignore (CONST-033 compliant, unchanged)	/etc/systemd/logind.conf
CONST-033 challenges	Not re-run this incident (already verified clean/hardened during BOB-116; no regression suspected — see docs/incidents/2026-08-18-perceived-forced-logout-2nd.md)	prior incident doc

Verdict: identical to BOB-116 — NO forbidden CONST-033 host-power path was invoked. The forced logout was NOT a suspend/hibernate/poweroff event. Host uptime is uninterrupted; this was a user@1000.service (user-session-scope) kill, not a host power-state transition.

What killed `user@1000.service`?

§11.4.6 HONEST BOUNDARY (carried forward from BOB-116, unchanged): the mechanism that delivered SIGKILL to `user@1000.service` at 23:45:49 is UNCONFIRMED.

As with BOB-116, the systemd PID 1 journal shows the *receipt* of SIGKILL by the main process and the subsequent cascade kill of every child process in the scope, but no kill-audit event attributing the *sender*. The same candidates from BOB-116 remain ruled out by direct evidence this incident:

- **Kernel OOM-killer:** journal clean, zero oom-kill markers.
- **systemd-oomd:** zero log entries in the incident window (would have logged a “Killed unit” message if triggered).
- **Lid-close suspend:** no lid events at all in this incident (unlike BOB-116) — `HandleLidSwitch=ignore` remains compliant regardless.
- **Manual `systemctl stop user@1000`:** no such event in the journal.

The candidates BOB-116 left open (systemd-oomd stealth kill, GDM/GNOME watchdog, third-party monitor escalation, cross-session kill from a sibling project) remain open and are **not** independently re-investigated in this incident — that deep forensic work is explicitly out of scope for this document (BOB-116’s own followup, PENDING_FORENSICS Task #79 for SIGKILL-source attribution, still owns that work; this is the *3rd data point* feeding it, not a duplicate investigation).

Contributing factors CONFIRMED

1. **qbittorrent-proxy health-probe `ConnectionResetError` cascade** — the socket-server handling the proxy’s health endpoint logged a `ConnectionResetError: [Errno 104] Connection reset by peer` roughly every 30 seconds continuously through the ~7-minute window captured (23:38–23:45), with the *last* log line at 23:45:44.737 — **5 seconds before the kill**. This is corroborating evidence of the same class §12.12/BOB-116 documented (a probe-vs-process contention signature), but is explicitly **not** established as causal — it is a health-probe *client* disconnecting mid-request against a server that was itself about to lose its process tree, which is consistent with either direction of causality (the probe client dying because resources were already tight, or the probe traffic itself being unrelated background noise that happened to be running when the kill landed). Filed as an observed correlation, not a cause.
2. **All four boba-stack containers were re-created (not merely restarted) 3 seconds after the kill**, before any human session existed to have issued a `podman/start.sh` command. This means the containers’ own supervision

(whatever launches/restarts them — outside the scope of this incident to trace exactly which unit did it) survived independently of `user@1000.service`'s death and self-healed within seconds. This is a **positive** finding (the boba stack self-recovers fast) but is also new information: it confirms the containers do **not** live inside the killed scope in a way that requires a human relogin to restart them — whatever killed `user@1000.service` did not also permanently take down the product's own containers.

3. **This is the SAME incident class recurring a 3rd time on this host** (2026-07-07 → §12.12 anchor; 2026-08-18 20:50:59 → BOB-116; 2026-08-18 23:45:49 → this incident, BOB-120), roughly 2h55m after the previous occurrence, on the same host, same session lineage. The recurrence rate (2 occurrences in under 3 hours) is itself evidence that whatever contributing pressure exists was not resolved by BOB-116's remediation.

Architectural finding: the preventive monitor lives inside the scope it protects

BOB-116's remediation (task #77) landed `boba-resource-pressure-check.timer` — an hourly `systemd --user` timer running `resource_pressure_signature_challenge.sh` to proactively detect the resource-pressure signatures (runaway RSS, thread-ceiling exhaustion, EAGAIN cascades, PSI pressure, pathological regex processes) BOB-116 found preceding the kill. This incident surfaces a real, previously-undocumented limitation of that design:

- The timer unit lives at `~/.config/systemd/user/boba-resource-pressure-check.timer`, loaded and run by the **user's own systemd --user manager instance** — which is itself a child of `user@1000.service`. **When `user@1000.service` dies, the timer, its triggered service, and any in-flight or about-to-fire probe die with it**, in the same SIGKILL cascade that kills everything else in the scope (the process list above shows `systemd[1665349]`, the user-manager PID that was hosting the timer, as itself belonging to the killed session).
- Captured fire history (`docs/qa/BOB-120/timer_fire_history.log`) shows the **last completed run before the kill finished at 22:57:58** — 47 minutes and 51 seconds before the 23:45:49 kill. Under the timer's own schedule (`OnUnitActiveSec=1hour`, `RandomizedDelaySec=5min`, measured from the *end* of the previous run), the next fire was due somewhere in the window **23:52:58-24:02:58** — i.e., the probe had **not yet reached its next scheduled activation** at the moment of the kill. This incident does **not** confirm the originally-hypothesized "the timer was overdue and silently missed its 23:42:38 fire"

framing — the real captured fire history shows the timer was simply not yet due, not that it missed a scheduled fire. That earlier hypothesis is corrected here per §11.4.6: it does not survive contact with the real `journalctl --user` data.

- The **real, confirmed** architectural gap is narrower but still genuine: because the monitor's own liveness is coupled to the exact scope it is meant to protect, (a) it provides no coverage for the final stretch of any hour between its own fires — up to ~1h05m of blind window per cycle (1h base + up to 5m jitter) — and (b) a kill that happens to land inside that blind window, however it arises, takes the monitor down with it rather than being caught mid-flight, so there is **no possibility of an in-scope self-monitor “seeing itself die.”** An out-of-scope watchdog (a `systemd` unit or process living *outside* `user@1000.service` — e.g. a root-owned system-level timer, or a unit under `--user` with `Delegate=/RemainAfterExit=` class isolation from the session scope) is required to close this class of gap entirely; the current design cannot, by construction, do so.
- **Post-incident observation** (captured live, `docs/qa/BOB-120/timer_fire_history.log`): after the new session started at 23:49:00, `boba-resource-pressure-check.timer` shows enabled + active (started 23:49:00) but `systemctl --user list-timers` reports NEXT: - (no next-fire time computed) and `systemctl --user status` reports Trigger: n/a, both captured ~7 minutes into the new session. Whether this resolves on its own once `OnBootSec=5min` elapses, or is itself a latent gap in the unit's post-restart re-arming, is **not confirmed** in this incident and is tracked as a followup rather than asserted as a finding — the honest state is “observed uncertain, not yet resolved to a fire or a genuine stuck-timer confirmation.”

This finding is filed as **BOB-120** (Critical, Queued — not closed in this commit, since closing it requires the actual architectural fix: an out-of-user-scope watchdog, tracked separately as the referenced follow-up task).

Actions taken (this incident)

1. **BOB-117 partial fix recovery** — already landed as commit 111e584 prior to this investigation starting (recovered from the stopped subagent ab2a34107); confirmed present at HEAD, not re-done here.
2. **This incident document** — `docs/incidents/2026-08-18-3rd-forced-logout.md`.
3. **Evidence capture** under `docs/qa/BOB-120/`:
 - `journalctl_23-42_to_23-46.log` — full system journal for the incident window.
 - `timer_fire_history.log` — the preventive timer's unit files, full 2026-08-18 fire history, and post-incident `list-timers/status` snapshots.

- `qbittorrent_proxy_socketserver_traces.log` — the health-probe `ConnectionResetError` cascade captured from `podman logs qbittorrent-proxy`.
- 4. **Memory playbook updated** — `forced_logout_incidents.md` gains this 3rd occurrence row and the “preventive-timer-inside-user-slice” architectural gap as a documented, named limitation (not a silent absorption).
- 5. **docs/QA_DISCOVERY_LEDGER.md** — new entry `FORCED-LOGOUT-2026-08-18-3RD` per §11.4.238, citing BOB-120 and the coverage-escape audit for why the automated regime (i.e., the very monitor built in response to BOB-116) still could not have caught this one.
- 6. **BOB-120 filed** in `docs/workable_items.db` — `Type=Bug`, `Severity=Critical`, `Status=Queued` (left open; closing requires the actual out-of-scope-watchdog architectural fix, not just this documentation).

Honest boundary (§11.4.6)

- The SIGKILL source remains UNCONFIRMED for all three incidents to date. This document does not claim to have identified it.
 - The “preventive timer inside `user.slice`” finding is a genuine, previously-undocumented architectural limitation of the BOB-116 remediation — it is not a bluff-cover for an unexplained kill; it is a real gap that would exist and matter even if the SIGKILL source were fully attributed tomorrow, because the monitor cannot outlive the scope it watches.
 - The originally-circulated “next fire was due at 23:42:38 and was silently missed” hypothesis does **not** hold up against the real captured fire history (last completed run 22:57:58, next due no earlier than 23:52:58) and is explicitly corrected here rather than repeated as fact.
 - Whether the `qbittorrent-proxy` `ConnectionResetError` cascade is causally related to the kill, or an independent symptom of the same underlying pressure, or unrelated noise, is **not** established — it is reported as a time-correlated observation only.
-

RETROSPECTIVE §11.4.6 CORRECTION — RESOLVED via BOB-126 (added 2026-08-19T17:32:00Z)

This doc's original hypothesis (PAM/Linger contradiction and/or various downstream mechanisms) was **WRONG**. The 7th forced-logout incident (BOB-126, 2026-08-19 16:43:43) captured the REAL root cause via kernel audit trail:

```
audit[399861]: SYSCALL syscall=62 a0=ffffffff a1=9
pid=399861 auid=1000 comm="pytest" exe="/usr/bin/python3.14"
key="sigkill_investigation"
```

A pytest process called `kill(-1, SIGKILL)`. Root cause: `tests/unit/merge_service/test_deadline_tunable.py::test_deadline_hit_flag_true_when_readline_times_out` created `AsyncMock()` without setting `mock.pid` as `int`. `Production_search_public_tracker` called `os.killpg(os.getpgid(proc.pid), SIGKILL)`. `MagicMock.__int__` defaults to `1`, so `os.getpgid(1) == 1` → `os.killpg(1, SIGKILL)` → `glibc` → `kill(-1, SIGKILL) = SIGKILL` every UID-1000 process. Bug existed since 2026-04-24 (~4 months).

PAM session_close was a DOWNSTREAM effect, not the initiator. The Linger=yes protection was irrelevant — the SIGKILL came from userspace (a UID-1000 process signaling its own UID-group), not from systemd's session lifecycle.

Fix chain:

- ad4b46a — boba: `search.py` int-guard + test hardening + §11.4.115 RED regression guard
- 502586c — constitution: NEW universal §11.4.263 anchor covering Python/Go/Rust/Bash/C
- bf01cf3 — boba: constitution pointer bump
- e984f4b — boba: `workable_items.db` chain-close + regenerated docs

Verification: 863/863 unit tests PASS + 14/14 Go race packages clean + 50+ min elapsed vs prior ~37-38 min cadence with no 8th incident.

§11.4.238 escape audit logged in `docs/QA_DISCOVERY_LEDGER.md` Rev 12.

See `docs/incidents/2026-08-19-6th-forced-logout.md` (Rev 4+) for the full attribution.